

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.22

CLIENT-SIDE DATABASE

 Copy as Markdown

Client-side database

The client-side database stores data on the device your app runs on, such as a phone or a browser, in a SQLite database. You query it with the same generated model methods you use on the server. This is useful for local caches, offline data, and state that belongs to the app rather than the server.

Enable the client-side database

A table model opts into the device database with the `database` keyword. Setting it to `client` generates the table only on the device, and `all` generates it on both the server and the device. The full value set is covered on the [Tables](#) page.

```
class: Company
table: company
database: client
```

When at least one table model has `database: client` or `database: all`, the generated `Client` class gets a `createSession` method that opens a SQLite database file and returns a `ClientDatabaseSession`.

INFO

The generated code depends on the `serverpod_database` package. Add it to your client package's `pubspec.yaml`.

Create a session

On Flutter, resolve a file path and call `createSession`:



```

import 'package:flutter/foundation.dart';
import 'package:flutter/widgets.dart';
import 'package:path/path.dart' as p;
import 'package:path_provider/path_provider.dart';
import 'package:my_project_client/my_project_client.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  // Point the client at your server as usual.
  final client = Client('http://localhost:8080/');

  // Resolve the database path.
  final path = await resolveDatabasePath('app.db');

  // Create the session to use in database operations.
  final session = await client.createSession(path, isDebugMode: kDebugMode);
}

Future<String> resolveDatabasePath(String fileName) async {
  if (kIsWeb) return fileName;
  final dir = await getApplicationSupportDirectory();
  return p.join(dir.path, fileName);
}

```

Creating the session opens the database file, so keep one session for the lifetime of the app, for example in your state management, instead of creating a new one per operation. Call `close()` on the session to close the underlying database when you are done with it. On the web there is no file system, which is why the example passes a bare name instead of a path.

NOTE

On the web, SQLite runs as WebAssembly in a web worker. Your Flutter app's `web/` directory needs two files, which the `sqlite_async` web setup links to:

- `sqlite3.wasm`, from the release that matches the `sqlite3` version in your `pubspec.lock`. Without it, `createSession` fails.
- `db_worker.js`, from the release that matches the `sqlite_async` version in your `pubspec.lock`. Without it, the database runs on the page's main thread and is not safe to use from several tabs.

INFO

On native platforms, SQLite runs in WAL mode, so `<path>-shm` and `<path>-wal` files may exist next to the database file while the session is open.

Run migrations on the device

Client tables get their own migrations. Whenever a migration is created for a project with client database tables, matching SQLite migrations are also generated under the client package's `lib/migrations/` directory.

By default, `createSession` applies pending client migrations when it opens the database (`runMigrations: true`). Passing `false` skips them, which can leave the database out of sync with your models, so only do so if you apply migrations through another code path.

With `isDebugMode: true`, the database integrity is verified after migrations are applied. On SQLite this includes a foreign key check that throws a `SQLiteMigrationForeignKeyViolationException` if it finds violating rows. See [Database exceptions](#). On Flutter, pass `kDebugMode` so the verification runs in debug builds only.

Use the database

The session works like a server-side session in database calls. Pass it to the generated model methods:

```
var companies = await Company.db.find(
  session,
  where: (t) => t.name.like('%Ltd'),
  orderBy: (t) => t.name,
);
```

CRUD, [filtering](#), [sorting](#), [pagination](#), [relations](#), and [transactions](#) all work as they do on the server. Because the database is SQLite, the SQLite-specific behaviors apply:

- `like` and `ilike` are both case-insensitive for ASCII characters. See [filtering](#).
- Transaction isolation levels are ignored, and transactions always run serialized. See [transactions](#).
- [Row locking](#) calls do nothing, since SQLite allows only one write transaction at a time.
- [Vector](#) queries are not supported, and [geography](#) values round-trip through CRUD but throw on spatial query operations.
- Column types and encodings differ from Postgres. See [Field types](#).

Sync tables with the server

WARNING

Offline sync is experimental. The `serverpod_offline_sync` module is still in development and not yet ready for production use.

A table with `database: sync` behaves like `database: all` and is additionally kept in sync between the device and the server by the `serverpod_offline_sync` module. Changes made while offline are merged on the next sync. The module tracks every operation and resolves conflicts itself, so your code never intervenes.

Enable the experimental feature and register the module in `config/generator.yaml` (or pass `--experimental-features databaseSync` on the command line):

```
modules:
  serverpod_offline_sync:
    nickname: offline_sync

experimental_features:
  databaseSync: true
```

Add `serverpod_offline_sync_server` to the server package and `serverpod_offline_sync_client` to the client package. Then mark the model. Sync tables need a UUID primary key. Serverpod adds a `spaceId` field for you, which links each row to the space that owns it: the user's personal space or a shared space.

```
class: Person
table: person
database: sync
fields:
  id: UuidValue?, defaultPersist=random_v7
  name: String
```

Code generation wires the sync engine into the generated `Serverpod` class, so the server needs no further changes. On the device, open the database with `createSyncSession` instead of `createSession`, then sync once or keep syncing:

```
final session = await client.createSyncSession(
  path,
  isDebugMode: kDebugMode,
  persistentUserId: persistentUserId,
);

await client.offlineSync.syncOnce(session);
// Or keep the device and the server in sync while the app runs:
final syncSession = client.offlineSync.syncContinuously(session);
```

The `createSyncSession` method returns an `OfflineSyncDatabaseSession` and takes the same `runMigrations` and `isDebugMode` parameters as `createSession`. The user must be signed in before syncing, or the sync calls fail as unauthorized. For personal and shared spaces, `persistentUserId`, and the rest of the API, see the `serverpod_offline_sync` README.

